

Time_Series_Analysis_Complete

October 7, 2025

1 Time Series Energy Consumption Analysis - Complete Workflow

This notebook performs the complete analysis: 1. Data Preprocessing 2. Billing Error Detection 3. Monthly Forecasting by Site 4. Quarterly Total Forecasting

Run all cells in order to generate all results.

1.1 Setup and Configuration

```
[2]: import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tools.sm_exceptions import ConvergenceWarning
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Configuration
MIN_DATA_POINTS = 12
FORECAST_HORIZON = 1
QUARTERS_TO_FORECAST = 4

print("Libraries loaded successfully")
print(f"Configuration:")
print(f"  Minimum data points for forecasting: {MIN_DATA_POINTS} months")
print(f"  Monthly forecast horizon: {FORECAST_HORIZON} month")
print(f"  Quarterly forecast horizon: {QUARTERS_TO_FORECAST} quarters")
```

Libraries loaded successfully

Configuration:

Minimum data points for forecasting: 12 months

Monthly forecast horizon: 1 month

Quarterly forecast horizon: 4 quarters

1.1.1 Load Excel Data

```
[3]: print("Loading Excel data...")
df_raw = pd.read_excel('Data_Projet.xlsx')

print(f"Original data shape: {df_raw.shape}")
print(f"Column name: {df_raw.columns[0]}")
print(f"\nFirst 5 rows:")
df_raw.head()
```

Loading Excel data...

Original data shape: (54131, 1)

Column name: SITE_NAME|INVOICE_NAME|BEGIN|END|KWH_PERIOD|PERIOD_EUR

First 5 rows:

```
[3]: SITE_NAME|INVOICE_NAME|BEGIN|END|KWH_PERIOD|PERIOD_EUR
0  12345_GEO (0001BB)|1300082549|01/01/2019|31/01...
1  12345_GEO (0001BB)|1300082549|01/02/2019|28/02...
2  12345_GEO (0001BB)|1300082549|01/03/2019|31/03...
3  12345_GEO (0001BB)|1300082549|01/04/2019|30/04...
4  12345_GEO (0001BB)|1300082549|01/05/2019|31/05...
```

1.1.2 Split Pipe-Delimited Column

```
[4]: # Extract column names
col_names = df_raw.columns[0].split('|')
print(f"Column names: {col_names}")

# Split the data
df = df_raw[df_raw.columns[0]].str.split('|', expand=True)
df.columns = col_names

print(f"\nData after splitting:")
df.head(10)
```

Column names: ['SITE_NAME', 'INVOICE_NAME', 'BEGIN', 'END', 'KWH_PERIOD', 'PERIOD_EUR']

Data after splitting:

```
[4]:      SITE_NAME INVOICE_NAME      BEGIN      END KWH_PERIOD \
0  12345_GEO (0001BB)  1300082549  01/01/2019  31/01/2019      3 094
1  12345_GEO (0001BB)  1300082549  01/02/2019  28/02/2019      2 767
2  12345_GEO (0001BB)  1300082549  01/03/2019  31/03/2019      3 059
3  12345_GEO (0001BB)  1300082549  01/04/2019  30/04/2019      2 761
4  12345_GEO (0001BB)  1300082549  01/05/2019  31/05/2019      2 844
5  12345_GEO (0001BB)  1300082549  01/06/2019  30/06/2019      3 009
6  12345_GEO (0001BB)  1300082549  01/07/2019  31/07/2019      3 016
```

7	12345_GEO (0001BB)	1300082549	01/08/2019	31/08/2019	3 089
8	12345_GEO (0001BB)	#	01/09/2019	30/09/2019	2 914
9	12345_GEO (0001BB)	1300082549	01/10/2019	31/10/2019	2 963

	PERIOD_EUR
0	381,40
1	345,87
2	378,18
3	340,84
4	349,26
5	369,19
6	371,20
7	380,42
8	361,36
9	366,70

1.1.3 Convert Data Types

```
[5]: # Function to clean numeric values
def clean_numeric(x):
    if pd.isna(x):
        return np.nan
    x = str(x).replace(' ', '').replace(',', '.')
    try:
        return float(x)
    except:
        return np.nan

# Convert numeric columns
df['KWH_PERIOD'] = df['KWH_PERIOD'].apply(clean_numeric)
df['PERIOD_EUR'] = df['PERIOD_EUR'].apply(clean_numeric)
print("Numeric columns converted")

# Convert date columns
df['BEGIN'] = pd.to_datetime(df['BEGIN'], format='%d/%m/%Y', errors='coerce')
df['END'] = pd.to_datetime(df['END'], format='%d/%m/%Y', errors='coerce')
print("Date columns converted")

print(f"\nData types:")
print(df.dtypes)
```

Numeric columns converted

Date columns converted

Data types:

SITE_NAME	object
INVOICE_NAME	object
BEGIN	datetime64[ns]

```

END                datetime64[ns]
KWH_PERIOD         float64
PERIOD_EUR         float64
dtype: object

```

1.1.4 Clean Data and Calculate Period Length

```

[6]: # Filter out zero consumption
df_filtered = df[df['KWH_PERIOD'] > 0].copy()
print(f"Records after removing zero consumption: {len(df_filtered)} (removed_
↳ {len(df) - len(df_filtered)})")

# Replace empty strings and # with NaN
df_filtered = df_filtered.replace(['', '#'], np.nan)

# Calculate billing period length in months
df_filtered['month_diff'] = ((df_filtered['END'] - df_filtered['BEGIN']).dt.
↳ days / 30).round().astype(int)

print(f"\nUnique billing period lengths (months):_
↳ {sorted(df_filtered['month_diff'].unique())}")
print(f"\nDistribution of period lengths:")
print(df_filtered['month_diff'].value_counts().sort_index())

```

Records after removing zero consumption: 54073 (removed 58)

Unique billing period lengths (months): [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 16, 19, 20, 21, 27, 30, 31, 37, 40]

Distribution of period lengths:

month_diff	
0	70
1	50153
2	109
3	1439
4	52
5	34
6	237
7	41
8	20
9	180
10	112
11	116
12	1492
13	6
16	1
19	2
20	3

```

21      1
27      1
30      1
31      1
37      1
40      1
Name: count, dtype: int64

```

1.1.5 Create Monthly Dataset

```

[7]: # Filter to only 1-month periods
df_monthly = df_filtered[df_filtered['month_diff'] == 1].copy()

# Calculate price per kWh
df_monthly['price_per_kwh'] = df_monthly['PERIOD_EUR'] / \
    df_monthly['KWH_PERIOD']

total_sites = df_filtered['SITE_NAME'].nunique()
monthly_sites = df_monthly['SITE_NAME'].nunique()

print(f"Records with 1-month periods: {len(df_monthly)}")
print(f"Total unique sites: {total_sites}")
print(f"Sites with monthly billing: {monthly_sites}")
print(f>Date range: {df_monthly['BEGIN'].min()} to {df_monthly['BEGIN'].max()}")
print(f"\nAverage consumption: {df_monthly['KWH_PERIOD'].mean():.2f} kWh")
print(f"Average cost: {df_monthly['PERIOD_EUR'].mean():.2f} EUR")
print(f"Average price per kWh: {df_monthly['price_per_kwh'].mean():.4f} EUR")

df_monthly.head()

```

```

Records with 1-month periods: 50153
Total unique sites: 2441
Sites with monthly billing: 1719
Date range: 2018-12-03 00:00:00 to 2021-11-12 00:00:00

Average consumption: 3125.73 kWh
Average cost: 381.94 EUR
Average price per kWh: 0.1310 EUR

```

```

[7]:      SITE_NAME  INVOICE_NAME      BEGIN      END  KWH_PERIOD  \
0  12345_GEO (0001BB)  1300082549  2019-01-01  2019-01-31      3094.0
1  12345_GEO (0001BB)  1300082549  2019-02-01  2019-02-28      2767.0
2  12345_GEO (0001BB)  1300082549  2019-03-01  2019-03-31      3059.0
3  12345_GEO (0001BB)  1300082549  2019-04-01  2019-04-30      2761.0
4  12345_GEO (0001BB)  1300082549  2019-05-01  2019-05-31      2844.0

      PERIOD_EUR  month_diff  price_per_kwh
0          381.40          1          0.123271

```

1	345.87	1	0.124998
2	378.18	1	0.123629
3	340.84	1	0.123448
4	349.26	1	0.122806

1.1.6 Initialize Error Log

```
[8]: error_log = pd.DataFrame(columns=['SITE_NAME', 'ERROR_TYPE', 'BEGIN', 'END',
                                     'KWH_PERIOD', 'PERIOD_EUR', 'DESCRIPTION',
                                     'SEVERITY'])

print("Error log initialized")
```

Error log initialized

1.1.7 Error Type 1: Irregular Billing Periods

```
[9]: print("Checking for irregular billing periods...")

irregular = df_filtered[df_filtered['month_diff'] != 1].copy()
irregular['ERROR_TYPE'] = 'IRREGULAR_PERIOD'
irregular['DESCRIPTION'] = irregular['month_diff'].apply(
    lambda x: f"Billing period spans {x} months instead of 1 month"
)
irregular['SEVERITY'] = irregular['month_diff'].apply(
    lambda x: 'HIGH' if (x == 0 or x >= 12) else ('MODERATE' if x >= 3 else
    'LOW')
)

error_log = pd.concat([error_log, irregular[['SITE_NAME', 'ERROR_TYPE',
    'BEGIN', 'END',
    'KWH_PERIOD', 'PERIOD_EUR',
    'DESCRIPTION', 'SEVERITY']]])

print(f"Found {len(irregular)} irregular billing periods")
```

Checking for irregular billing periods...
Found 3920 irregular billing periods

1.1.8 Error Type 2: Duplicate Billing Periods

```
[10]: print("Checking for duplicate billing periods...")

duplicates = df_monthly[df_monthly.duplicated(subset=['SITE_NAME', 'BEGIN'],
    keep=False)].copy()
duplicates['ERROR_TYPE'] = 'DUPLICATE_PERIOD'
duplicates['DESCRIPTION'] = duplicates['BEGIN'].apply(
    lambda x: f"Duplicate entry for period starting {x.strftime('%Y-%m-%d')}"
)
```

```

)
duplicates['SEVERITY'] = 'HIGH'

error_log = pd.concat([error_log, duplicates[['SITE_NAME', 'ERROR_TYPE', '
↳ 'BEGIN', 'END',
                                                    'KWH_PERIOD', 'PERIOD_EUR', '
↳ 'DESCRIPTION', 'SEVERITY']]])

print(f"Found {len(duplicates)} duplicate billing periods")

```

Checking for duplicate billing periods...
Found 277 duplicate billing periods

1.1.9 Error Type 3: Missing Billing Periods

```

[11]: print("Checking for missing billing periods (this may take a minute)...")

missing_periods = []
sites = df_monthly['SITE_NAME'].unique()

for idx, site in enumerate(sites):
    if (idx + 1) % 500 == 0:
        print(f"  Processed {idx + 1}/{len(sites)} sites...")

    site_data = df_monthly[df_monthly['SITE_NAME'] == site].sort_values('BEGIN')

    if len(site_data) >= 2:
        for i in range(len(site_data) - 1):
            expected_next = site_data.iloc[i]['BEGIN'] + pd.DateOffset(months=1)
            actual_next = site_data.iloc[i + 1]['BEGIN']

            if expected_next != actual_next:
                gap_months = (actual_next.year - expected_next.year) * 12 +
↳ (actual_next.month - expected_next.month)

                missing_periods.append({
                    'SITE_NAME': site,
                    'ERROR_TYPE': 'MISSING_PERIOD',
                    'BEGIN': expected_next,
                    'END': expected_next + pd.DateOffset(months=1) - pd.
↳ DateOffset(days=1),
                    'KWH_PERIOD': np.nan,
                    'PERIOD_EUR': np.nan,
                    'DESCRIPTION': f"Missing billing data for {expected_next.
↳ strftime('%Y-%m')} ({gap_months} month gap)",
                    'SEVERITY': 'HIGH' if gap_months > 3 else 'MODERATE'
                })

```

```

if missing_periods:
    missing_df = pd.DataFrame(missing_periods)
    error_log = pd.concat([error_log, missing_df])

print(f"Found {len(missing_periods)} missing billing periods")

```

Checking for missing billing periods (this may take a minute)...

Processed 500/1719 sites...

Processed 1000/1719 sites...

Processed 1500/1719 sites...

Found 434 missing billing periods

1.1.10 Error Type 4: Consumption Anomalies

```

[12]: print("Checking for consumption anomalies...")

# Calculate statistics per site
site_stats = df_monthly.groupby('SITE_NAME')['KWH_PERIOD'].agg(['mean', 'std',
    ↪ 'count']).reset_index()
site_stats = site_stats[site_stats['count'] >= 6]

# Merge and calculate z-scores
df_with_stats = df_monthly.merge(site_stats, on='SITE_NAME', suffixes=('_',
    ↪ '_site'))
df_with_stats['z_score'] = (df_with_stats['KWH_PERIOD'] -
    ↪ df_with_stats['mean']) / df_with_stats['std']

# Find anomalies (z-score > 3)
anomalies = df_with_stats[np.abs(df_with_stats['z_score']) > 3].copy()
anomalies['ERROR_TYPE'] = 'CONSUMPTION_ANOMALY'
anomalies['DESCRIPTION'] = anomalies.apply(
    lambda row: f"Consumption is {abs(row['z_score']):.1f} std deviations from
    ↪ site average (mean={row['mean']:.0f} kWh)",
    axis=1
)
anomalies['SEVERITY'] = anomalies['z_score'].apply(
    lambda x: 'HIGH' if abs(x) > 5 else ('MODERATE' if abs(x) > 3 else 'LOW')
)

error_log = pd.concat([error_log, anomalies[['SITE_NAME', 'ERROR_TYPE',
    ↪ 'BEGIN', 'END',
    ↪ 'KWH_PERIOD', 'PERIOD_EUR',
    ↪ 'DESCRIPTION', 'SEVERITY']]])

print(f"Found {len(anomalies)} consumption anomalies")

```

Checking for consumption anomalies...

Found 176 consumption anomalies

1.1.11 Error Type 5: Price Anomalies

```
[13]: print("Checking for price anomalies...")

overall_mean_price = df_monthly['price_per_kwh'].mean()
print(f"Overall average price per kWh: {overall_mean_price:.4f} EUR")

# Site-level price statistics
price_stats = df_monthly.groupby('SITE_NAME')['price_per_kwh'].agg(['mean',
    ↪ 'std', 'count']).reset_index()
price_stats = price_stats[price_stats['count'] >= 6]

df_with_price_stats = df_monthly.merge(price_stats, on='SITE_NAME',
    ↪ suffixes=('', '_site'))
df_with_price_stats['price_z_score'] = (df_with_price_stats['price_per_kwh'] -
    ↪ df_with_price_stats['mean']) / df_with_price_stats['std']

# Find price anomalies
price_anomalies = df_with_price_stats[
    (np.abs(df_with_price_stats['price_z_score']) > 3) |
    (df_with_price_stats['price_per_kwh'] < 0.05) |
    (df_with_price_stats['price_per_kwh'] > 0.30)
].copy()

price_anomalies['ERROR_TYPE'] = 'PRICE_ANOMALY'
price_anomalies['DESCRIPTION'] = price_anomalies.apply(
    lambda row: f"Price per kWh is {row['price_per_kwh']:.4f} EUR (site
    ↪ avg={row['mean']:.4f} EUR)",
    axis=1
)
price_anomalies['SEVERITY'] = price_anomalies.apply(
    lambda row: 'HIGH' if (row['price_per_kwh'] < 0.05 or row['price_per_kwh']
    ↪ > 0.30) else ('MODERATE' if abs(row['price_z_score']) > 4 else 'LOW'),
    axis=1
)

error_log = pd.concat([error_log, price_anomalies[['SITE_NAME', 'ERROR_TYPE',
    ↪ 'BEGIN', 'END',
    'KWH_PERIOD', 'PERIOD_EUR',
    ↪ 'DESCRIPTION', 'SEVERITY']]])

print(f"Found {len(price_anomalies)} price anomalies")
```

Checking for price anomalies...

Overall average price per kWh: 0.1310 EUR

Found 207 price anomalies

1.1.12 Error Type 6: Invalid Costs

```
[14]: print("Checking for invalid costs...")

invalid_costs = df_monthly[(df_monthly['PERIOD_EUR'] <= 0) |
    ↪(df_monthly['PERIOD_EUR'].isna())].copy()
invalid_costs['ERROR_TYPE'] = 'INVALID_COST'
invalid_costs['DESCRIPTION'] = invalid_costs['PERIOD_EUR'].apply(
    ↪lambda x: "Cost is missing" if pd.isna(x) else f"Cost is {x} EUR (should be
    ↪positive)"
)
invalid_costs['SEVERITY'] = 'HIGH'

error_log = pd.concat([error_log, invalid_costs[['SITE_NAME', 'ERROR_TYPE',
    ↪'BEGIN', 'END',
    ↪'KWH_PERIOD', 'PERIOD_EUR',
    ↪'DESCRIPTION', 'SEVERITY']]])

print(f"Found {len(invalid_costs)} invalid cost records")
```

Checking for invalid costs...

Found 0 invalid cost records

1.1.13 Billing Error Summary

```
[15]: print("="*60)
print("BILLING ERROR SUMMARY")
print("="*60)

total_sites_analyzed = df_monthly['SITE_NAME'].nunique()
sites_with_errors = error_log['SITE_NAME'].nunique()

print(f"Total sites analyzed: {total_sites_analyzed}")
print(f"Sites with errors: {sites_with_errors}")
print(f"Percentage with errors: {100 * sites_with_errors / total_sites_analyzed:
    ↪.1f}%\n")

print("Error breakdown by type:")
print(error_log['ERROR_TYPE'].value_counts())
print()

print("Error breakdown by severity:")
print(error_log['SEVERITY'].value_counts())
print("="*60)
```

```
# Save error reports
error_log.to_csv('billing_errors_report.csv', index=False)
print(f"\nSaved: billing_errors_report.csv ({len(error_log)} errors)")

high_severity = error_log[error_log['SEVERITY'] == 'HIGH']
high_severity.to_csv('billing_errors_high_severity.csv', index=False)
print(f"Saved: billing_errors_high_severity.csv ({len(high_severity)} high_
severity errors)")
```

BILLING ERROR SUMMARY

Total sites analyzed: 1719
 Sites with errors: 1398
 Percentage with errors: 81.3%

Error breakdown by type:

```
ERROR_TYPE
IRREGULAR_PERIOD      3920
MISSING_PERIOD        434
DUPLICATE_PERIOD      277
PRICE_ANOMALY         207
CONSUMPTION_ANOMALY   176
Name: count, dtype: int64
```

Error breakdown by severity:

```
SEVERITY
MODERATE    2863
HIGH        1923
LOW         228
Name: count, dtype: int64
```

Saved: billing_errors_report.csv (5014 errors)

Saved: billing_errors_high_severity.csv (1923 high severity errors)

1.1.14 Define Forecasting Function

```
[16]: def forecast_site(site_name, df, forecast_periods=1):
        """
        Forecast energy consumption for a single site.
        Uses ARIMA for sites with enough data, seasonal naive for others.
        """
        try:
            site_data = df[df['SITE_NAME'] == site_name].sort_values('BEGIN').copy()
            site_data = site_data.set_index('BEGIN')['KWH_PERIOD']
```

```

n_obs = len(site_data)

# Insufficient data - use seasonal naive
if n_obs < MIN_DATA_POINTS:
    last_year_idx = max(0, n_obs - 12)
    forecast_value = site_data.iloc[last_year_idx]
    lower_bound = forecast_value * 0.8
    upper_bound = forecast_value * 1.2
    forecast_date = site_data.index[-1] + pd.DateOffset(months=1)

    return {
        'SITE_NAME': site_name,
        'FORECAST_MONTH': forecast_date,
        'YHAT': forecast_value,
        'YHAT_LOWER': lower_bound,
        'YHAT_UPPER': upper_bound,
        'MODEL_USED': 'SEASONAL_NAIVE',
        'DATA_POINTS': n_obs,
        'VALIDATION_MAE': np.nan
    }

# Train-test split
n_test = min(6, int(n_obs * 0.2))
n_train = n_obs - n_test
train = site_data.iloc[:n_train]
test = site_data.iloc[n_train:]

# Try ARIMA
try:
    model = ARIMA(train, order=(1,0,1), seasonal_order=(0,0,0,0))
    fitted = model.fit()
    test_forecast = fitted.forecast(steps=len(test))
    mae = np.mean(np.abs(test.values - test_forecast.values))

    final_model = ARIMA(site_data, order=(1,0,1),
↪seasonal_order=(0,0,0,0))
    final_fitted = final_model.fit()
    final_forecast = final_fitted.forecast(steps=forecast_periods)
    pred_int = final_fitted.get_forecast(steps=forecast_periods).
↪conf_int()

    forecast_date = site_data.index[-1] + pd.DateOffset(months=1)

    return {
        'SITE_NAME': site_name,
        'FORECAST_MONTH': forecast_date,
        'YHAT': final_forecast.iloc[0],

```

```

        'YHAT_LOWER': pred_int.iloc[0, 0],
        'YHAT_UPPER': pred_int.iloc[0, 1],
        'MODEL_USED': 'ARIMA',
        'DATA_POINTS': n_obs,
        'VALIDATION_MAE': mae
    }
except:
    # Try exponential smoothing
    try:
        model = ExponentialSmoothing(train, trend='add', seasonal=None)
        fitted = model.fit()
        test_forecast = fitted.forecast(steps=len(test))
        mae = np.mean(np.abs(test.values - test_forecast.values))

        final_model = ExponentialSmoothing(site_data, trend='add', ↵
seasonal=None)
        final_fitted = final_model.fit()
        final_forecast = final_fitted.forecast(steps=forecast_periods)

        forecast_value = final_forecast.iloc[0]
        std_error = site_data.std() * 0.3
        forecast_date = site_data.index[-1] + pd.DateOffset(months=1)

        return {
            'SITE_NAME': site_name,
            'FORECAST_MONTH': forecast_date,
            'YHAT': forecast_value,
            'YHAT_LOWER': forecast_value - 1.96 * std_error,
            'YHAT_UPPER': forecast_value + 1.96 * std_error,
            'MODEL_USED': 'EXP_SMOOTHING',
            'DATA_POINTS': n_obs,
            'VALIDATION_MAE': mae
        }
    except:
        pass

# Fallback to naive
forecast_value = site_data.iloc[-1]
forecast_date = site_data.index[-1] + pd.DateOffset(months=1)

return {
    'SITE_NAME': site_name,
    'FORECAST_MONTH': forecast_date,
    'YHAT': forecast_value,
    'YHAT_LOWER': forecast_value * 0.7,
    'YHAT_UPPER': forecast_value * 1.3,
    'MODEL_USED': 'NAIVE_FALLBACK',

```

```

        'DATA_POINTS': n_obs,
        'VALIDATION_MAE': np.nan
    }
except:
    return None

print("Forecasting function defined")

```

Forecasting function defined

1.1.15 Run Forecasts for All Sites

```

[17]: sites = df_monthly['SITE_NAME'].unique()
print(f"Running forecasts for {len(sites)} sites...")
print("This may take a few minutes...\n")

forecasts = []
for i, site in enumerate(sites):
    if (i + 1) % 100 == 0:
        print(f"Processed {i + 1}/{len(sites)} sites...")

    result = forecast_site(site, df_monthly, FORECAST_HORIZON)
    if result:
        forecasts.append(result)

forecasts_df = pd.DataFrame(forecasts)
print(f"\nForecasting complete! Generated {len(forecasts_df)} forecasts")

```

Running forecasts for 1719 sites...

This may take a few minutes...

```

Processed 100/1719 sites...
Processed 200/1719 sites...
Processed 300/1719 sites...
Processed 400/1719 sites...
Processed 500/1719 sites...
Processed 600/1719 sites...
Processed 700/1719 sites...
Processed 800/1719 sites...
Processed 900/1719 sites...
Processed 1000/1719 sites...
Processed 1100/1719 sites...
Processed 1200/1719 sites...
Processed 1300/1719 sites...
Processed 1400/1719 sites...
Processed 1500/1719 sites...
Processed 1600/1719 sites...
Processed 1700/1719 sites...

```

Forecasting complete! Generated 1719 forecasts

1.1.16 Monthly Forecast Summary

```
[18]: print("="*60)
print("MONTHLY FORECASTING SUMMARY")
print("="*60)

print(f"Total sites forecasted: {len(forecasts_df)}")
print(f"Average forecast (kWh): {forecasts_df['YHAT'].mean():.2f}")
print(f"Median forecast (kWh): {forecasts_df['YHAT'].median():.2f}")
print(f"Total forecasted consumption: {forecasts_df['YHAT'].sum():.0f} kWh\n")

print("Model distribution:")
print(forecasts_df['MODEL_USED'].value_counts())
print()

mae_stats = forecasts_df[forecasts_df['VALIDATION_MAE'].notna()]
if len(mae_stats) > 0:
    print(f"Validation MAE statistics ({len(mae_stats)} models):")
    print(f"  Mean MAE: {mae_stats['VALIDATION_MAE'].mean():.2f} kWh")
    print(f"  Median MAE: {mae_stats['VALIDATION_MAE'].median():.2f} kWh")

forecasts_df['CI_WIDTH'] = forecasts_df['YHAT_UPPER'] -
    forecasts_df['YHAT_LOWER']
print(f"\nAverage confidence interval width: {forecasts_df['CI_WIDTH'].mean():.2f} kWh")
print("="*60)

# Save forecasts
forecasts_df.to_csv('monthly_forecasts_by_site.csv', index=False)
print("\nSaved: monthly_forecasts_by_site.csv")
```

```
=====
MONTHLY FORECASTING SUMMARY
=====

Total sites forecasted: 1719
Average forecast (kWh): 2904.65
Median forecast (kWh): 2092.87
Total forecasted consumption: 4,993,093 kWh

Model distribution:
MODEL_USED
ARIMA          1424
SEASONAL_NAIVE 295
Name: count, dtype: int64
```

Validation MAE statistics (1424 models):

Mean MAE: 198.65 kWh

Median MAE: 72.52 kWh

Average confidence interval width: 731.78 kWh

=====

Saved: monthly_forecasts_by_site.csv

1.1.17 Prepare Historical Quarterly Data

```
[19]: # Add quarter information
df_monthly['YEAR'] = df_monthly['BEGIN'].dt.year
df_monthly['QUARTER'] = df_monthly['BEGIN'].dt.quarter
df_monthly['YEAR_QUARTER'] = df_monthly['YEAR'].astype(str) + '-Q' +
    ↪df_monthly['QUARTER'].astype(str)

# Aggregate to quarterly level
quarterly_data = df_monthly.groupby(['YEAR', 'QUARTER', 'YEAR_QUARTER']).agg({
    'KWH_PERIOD': 'sum',
    'PERIOD_EUR': 'sum',
    'SITE_NAME': 'count'
}).reset_index()
quarterly_data.columns = ['YEAR', 'QUARTER', 'YEAR_QUARTER', 'TOTAL_KWH',
    ↪'TOTAL_EUR', 'N_RECORDS']
quarterly_data = quarterly_data.sort_values(['YEAR', 'QUARTER'])

print("Historical quarterly consumption:")
print(quarterly_data)
```

Historical quarterly consumption:

	YEAR	QUARTER	YEAR_QUARTER	TOTAL_KWH	TOTAL_EUR	N_RECORDS
0	2018	4	2018-Q4	8389.0	1114.36	8
1	2019	1	2019-Q1	12253757.0	1411239.73	4105
2	2019	2	2019-Q2	12831294.0	1480030.23	4145
3	2019	3	2019-Q3	13367455.0	1544119.65	4211
4	2019	4	2019-Q4	12925912.0	1498192.14	4219
5	2020	1	2020-Q1	12936234.0	1680621.44	4239
6	2020	2	2020-Q2	13404364.0	1740233.58	4250
7	2020	3	2020-Q3	14067842.0	1818676.22	4259
8	2020	4	2020-Q4	13647094.0	1764032.95	4278
9	2021	1	2021-Q1	13439590.0	1636519.17	4367
10	2021	2	2021-Q2	13826598.0	1679738.16	4375
11	2021	3	2021-Q3	14229365.0	1725530.72	4378
12	2021	4	2021-Q4	9826621.0	1175335.74	3319

1.1.18 Approach 1: Bottom-Up (Aggregate Site Forecasts)

```
[20]: print("\nApproach 1: Bottom-Up from site forecasts")

last_date = df_monthly['BEGIN'].max()
print(f"Last observed period: {last_date}\n")

# Calculate seasonal patterns
monthly_totals = df_monthly.groupby('BEGIN')['KWH_PERIOD'].sum().reset_index()
monthly_totals['QUARTER'] = monthly_totals['BEGIN'].dt.quarter

quarterly_patterns = monthly_totals.groupby('QUARTER')['KWH_PERIOD'].mean().
    ↪reset_index()
quarterly_patterns.columns = ['QUARTER', 'AVG_MONTHLY_TOTAL']

total_monthly_forecast = forecasts_df['YHAT'].sum()
forecast_start_date = last_date + pd.DateOffset(months=1)

bottomup_forecasts = []

for q in range(1, QUARTERS_TO_FORECAST + 1):
    quarter_start_month = forecast_start_date.month + (q - 1) * 3
    quarter_num = ((quarter_start_month - 1) // 3) % 4 + 1
    year_num = forecast_start_date.year + ((forecast_start_date.month + (q - 1)
    ↪* 3 - 1) // 12)

    seasonal_factor = quarterly_patterns[quarterly_patterns['QUARTER'] ==
    ↪quarter_num]['AVG_MONTHLY_TOTAL'].values[0]
    seasonal_factor = seasonal_factor / quarterly_patterns['AVG_MONTHLY_TOTAL'].
    ↪mean()

    quarterly_forecast = total_monthly_forecast * 3 * seasonal_factor

    bottomup_forecasts.append({
        'YEAR': year_num,
        'QUARTER': quarter_num,
        'YEAR_QUARTER': f"{year_num}-Q{quarter_num}",
        'TOTAL_KWH_FORECAST': quarterly_forecast,
        'LOWER_BOUND': quarterly_forecast * 0.90,
        'UPPER_BOUND': quarterly_forecast * 1.10,
        'APPROACH': 'BOTTOM_UP'
    })

bottomup_df = pd.DataFrame(bottomup_forecasts)
print("Bottom-Up Forecasts:")
print(bottomup_df)
```

Approach 1: Bottom-Up from site forecasts
 Last observed period: 2021-11-12 00:00:00

Bottom-Up Forecasts:

	YEAR	QUARTER	YEAR_QUARTER	TOTAL_KWH_FORECAST	LOWER_BOUND	UPPER_BOUND	\
0	2021	4	2021-Q4	1.128370e+07	1.015533e+07	1.241207e+07	
1	2022	1	2022-Q1	1.224431e+07	1.101988e+07	1.346875e+07	
2	2022	2	2022-Q2	2.148964e+07	1.934068e+07	2.363861e+07	
3	2022	3	2022-Q3	1.489946e+07	1.340951e+07	1.638940e+07	

APPROACH
 0 BOTTOM_UP
 1 BOTTOM_UP
 2 BOTTOM_UP
 3 BOTTOM_UP

1.1.19 Approach 2: Top-Down (ARIMA on Quarterly Aggregates)

```
[21]: print("\nApproach 2: Top-Down ARIMA on quarterly aggregates")

ts_quarterly = quarterly_data.set_index(pd.
    ↳PeriodIndex(quarterly_data['YEAR_QUARTER'], freq='Q'))['TOTAL_KWH']

try:
    model = ARIMA(ts_quarterly, order=(1,0,1), seasonal_order=(1,0,0,4))
    fitted = model.fit()

    forecast = fitted.forecast(steps=QUARTERS_TO_FORECAST)
    pred_int = fitted.get_forecast(steps=QUARTERS_TO_FORECAST).conf_int()

    topdown_forecasts = []
    for i in range(QUARTERS_TO_FORECAST):
        forecast_period = forecast.index[i]
        year = forecast_period.year
        quarter = forecast_period.quarter

        topdown_forecasts.append({
            'YEAR': year,
            'QUARTER': quarter,
            'YEAR_QUARTER': f"{year}-Q{quarter}",
            'TOTAL_KWH_FORECAST': forecast.iloc[i],
            'LOWER_BOUND': pred_int.iloc[i, 0],
            'UPPER_BOUND': pred_int.iloc[i, 1],
            'APPROACH': 'TOP_DOWN_ARIMA'
        })

    topdown_df = pd.DataFrame(topdown_forecasts)
```

```

print("Top-Down Forecasts:")
print(topdown_df)

except Exception as e:
    print(f"ARIMA failed: {e}")
    print("Using exponential smoothing\n")

    model = ExponentialSmoothing(ts_quarterly, trend='add', seasonal='add',
seasonal_periods=4)
    fitted = model.fit()
    forecast = fitted.forecast(steps=QUARTERS_TO_FORECAST)
    std_error = ts_quarterly.std() * 0.15

    topdown_forecasts = []
    for i in range(QUARTERS_TO_FORECAST):
        forecast_period = forecast.index[i]
        year = forecast_period.year
        quarter = forecast_period.quarter

        topdown_forecasts.append({
            'YEAR': year,
            'QUARTER': quarter,
            'YEAR_QUARTER': f"{year}-Q{quarter}",
            'TOTAL_KWH_FORECAST': forecast.iloc[i],
            'LOWER_BOUND': forecast.iloc[i] - 1.96 * std_error,
            'UPPER_BOUND': forecast.iloc[i] + 1.96 * std_error,
            'APPROACH': 'TOP_DOWN_ETS'
        })

    topdown_df = pd.DataFrame(topdown_forecasts)
    print("Top-Down Forecasts:")
    print(topdown_df)

```

Approach 2: Top-Down ARIMA on quarterly aggregates

Top-Down Forecasts:

	YEAR	QUARTER	YEAR_QUARTER	TOTAL_KWH_FORECAST	LOWER_BOUND	UPPER_BOUND	\
0	2022	1	2022-Q1	1.133533e+07	8.575347e+06	1.409532e+07	
1	2022	2	2022-Q2	1.146855e+07	8.677453e+06	1.425965e+07	
2	2022	3	2022-Q3	1.157337e+07	8.762565e+06	1.438417e+07	
3	2022	4	2022-Q4	1.174205e+07	8.918730e+06	1.456537e+07	

APPROACH

0	TOP_DOWN_ARIMA
1	TOP_DOWN_ARIMA
2	TOP_DOWN_ARIMA
3	TOP_DOWN_ARIMA

1.1.20 Ensemble Forecast (Recommended)

```
[22]: all_quarterly_forecasts = pd.concat([bottomup_df, topdown_df],  
      ↪ ignore_index=True)  
  
ensemble_forecasts = all_quarterly_forecasts.groupby('YEAR_QUARTER').agg({  
    'TOTAL_KWH_FORECAST': 'mean',  
    'LOWER_BOUND': 'min',  
    'UPPER_BOUND': 'max'  
}).reset_index()  
ensemble_forecasts['APPROACH'] = 'ENSEMBLE'  
  
print("\nEnsemble Forecasts (average of both methods):")  
print(ensemble_forecasts)
```

Ensemble Forecasts (average of both methods):

	YEAR_QUARTER	TOTAL_KWH_FORECAST	LOWER_BOUND	UPPER_BOUND	APPROACH
0	2021-Q4	1.128370e+07	1.015533e+07	1.241207e+07	ENSEMBLE
1	2022-Q1	1.178982e+07	8.575347e+06	1.409532e+07	ENSEMBLE
2	2022-Q2	1.647910e+07	8.677453e+06	2.363861e+07	ENSEMBLE
3	2022-Q3	1.323641e+07	8.762565e+06	1.638940e+07	ENSEMBLE
4	2022-Q4	1.174205e+07	8.918730e+06	1.456537e+07	ENSEMBLE

1.1.21 Quarterly Forecast Summary

```
[23]: print("\n" + "="*60)
print("QUARTERLY FORECAST SUMMARY")
print("="*60)

forecast_total = ensemble_forecasts['TOTAL_KWH_FORECAST'].sum()
historical_avg_quarterly = quarterly_data['TOTAL_KWH'].mean()

print(f"Forecast period: {ensemble_forecasts['YEAR_QUARTER'].min()} to
↳{ensemble_forecasts['YEAR_QUARTER'].max()}")
print(f"Total 4-quarter forecast: {forecast_total:,.0f} kWh")
print(f"Average quarterly forecast: {forecast_total / 4:,.0f} kWh")
print(f"Historical quarterly average: {historical_avg_quarterly:,.0f} kWh")
print(f"Change from historical avg: {((forecast_total / 4) /
↳historical_avg_quarterly - 1) * 100:.1f}%")
print("="*60)

# Save forecasts
all_quarterly_forecasts.to_csv('quarterly_forecasts_all_approaches.csv',
↳index=False)
ensemble_forecasts.to_csv('quarterly_forecasts_ensemble.csv', index=False)
print("\nSaved: quarterly_forecasts_all_approaches.csv")
print("Saved: quarterly_forecasts_ensemble.csv")
```

```
=====
QUARTERLY FORECAST SUMMARY
=====
Forecast period: 2021-Q4 to 2022-Q4
Total 4-quarter forecast: 64,531,089 kWh
Average quarterly forecast: 16,132,772 kWh
Historical quarterly average: 12,058,809 kWh
Change from historical avg: 33.8%
=====

Saved: quarterly_forecasts_all_approaches.csv
Saved: quarterly_forecasts_ensemble.csv
```

1.1.22 Visualization

```
[24]: # Prepare plot data
historical_plot = quarterly_data[['YEAR_QUARTER', 'TOTAL_KWH']].copy()
forecast_plot = ensemble_forecasts[['YEAR_QUARTER', 'TOTAL_KWH_FORECAST']].
↳copy()
forecast_plot.columns = ['YEAR_QUARTER', 'TOTAL_KWH']

# Create plot
plt.figure(figsize=(12, 6))
```

```

hist_indices = range(len(historical_plot))
plt.plot(hist_indices, historical_plot['TOTAL_KWH'],
         marker='o', color='darkgreen', linewidth=2, markersize=8,
         label='Historical')

forecast_indices = range(len(historical_plot), len(historical_plot) +
                        len(forecast_plot))
plt.plot(forecast_indices, forecast_plot['TOTAL_KWH'],
         marker='o', color='blue', linewidth=2, markersize=8, label='Forecast')

plt.fill_between(forecast_indices,
                 ensemble_forecasts['LOWER_BOUND'],
                 ensemble_forecasts['UPPER_BOUND'],
                 alpha=0.2, color='blue')

all_quarters = list(historical_plot['YEAR_QUARTER']) +
               list(forecast_plot['YEAR_QUARTER'])
plt.xticks(range(len(all_quarters)), all_quarters, rotation=45, ha='right')
plt.xlabel('Quarter')
plt.ylabel('Total Consumption (kWh)')
plt.title('Quarterly Energy Consumption: Historical and Forecast')
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()

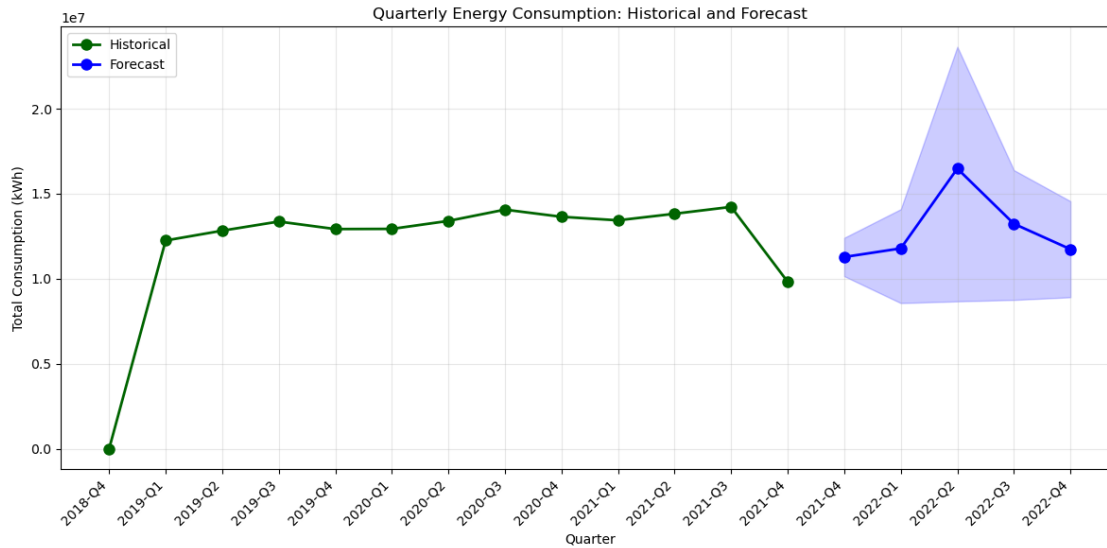
plt.savefig('quarterly_forecast_plot.pdf', dpi=300, bbox_inches='tight')
plt.savefig('quarterly_forecast_plot.png', dpi=300, bbox_inches='tight')
print("Saved: quarterly_forecast_plot.pdf")
print("Saved: quarterly_forecast_plot.png")

plt.show()

```

Saved: quarterly_forecast_plot.pdf

Saved: quarterly_forecast_plot.png



```
[25]: print("\n" + "="*70)
print("ANALYSIS COMPLETE - ALL OUTPUTS CREATED")
print("="*70)

print("\n1. BILLING ERROR DETECTION:")
print(f" - Total errors found: {len(error_log)}")
print(f" - Sites with errors: {sites_with_errors}/{total_sites_analyzed}")
print(f" - High severity errors: {len(high_severity)}")
print(" - Files: billing_errors_report.csv, billing_errors_high_severity.csv")

print("\n2. MONTHLY FORECASTS:")
print(f" - Sites forecasted: {len(forecasts_df)}")
print(f" - Total monthly forecast: {forecasts_df['YHAT'].sum():.0f} kWh")
print(f" - Average forecast per site: {forecasts_df['YHAT'].mean():.0f} kWh")
print(" - File: monthly_forecasts_by_site.csv")

print("\n3. QUARTERLY FORECASTS:")
print(f" - Forecast period: {ensemble_forecasts['YEAR_QUARTER'].min()} to {ensemble_forecasts['YEAR_QUARTER'].max()}")
print(f" - Total 4-quarter forecast: {forecast_total:.0f} kWh")
print(f" - Average quarterly: {forecast_total / 4:.0f} kWh")
print(" - Files: quarterly_forecasts_ensemble.csv, quarterly_forecast_plot.pdf")

print("\n" + "="*70)
print("All tasks completed successfully!")
print("="*70)
```

```
=====
ANALYSIS COMPLETE - ALL OUTPUTS CREATED
=====
```

1. BILLING ERROR DETECTION:

- Total errors found: 5014
- Sites with errors: 1398/1719
- High severity errors: 1923
- Files: billing_errors_report.csv, billing_errors_high_severity.csv

2. MONTHLY FORECASTS:

- Sites forecasted: 1719
- Total monthly forecast: 4,993,093 kWh
- Average forecast per site: 2905 kWh
- File: monthly_forecasts_by_site.csv

3. QUARTERLY FORECASTS:

- Forecast period: 2021-Q4 to 2022-Q4
- Total 4-quarter forecast: 64,531,089 kWh
- Average quarterly: 16,132,772 kWh
- Files: quarterly_forecasts_ensemble.csv, quarterly_forecast_plot.pdf

```
=====
All tasks completed successfully!
=====
```